Model-Based FPGA AI Accelerator Design Using Vitis AI with In-Depth Performance and Energy Efficiency Analysis

1st Gözde Özdil
Istanbul Technical University, Turkiye
Kuartis Technology, Turkiye
ozdil19@itu.edu.tr

2nd Berna Örs
Istanbul Technical University, Turkiye
orssi@itu.edu.tr

2024 32nd Telecommunications Forum (TELFOR) | 979-8-3503-9106-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/TELFOR63250.2024.10819065

***Abstract*—This paper presents a model-based design of AI accelerator following the Vitis TRD flow, implemented on the AMD Kria KV260 Vision AI Starter Kit. The ResNet-18 model, developed in PyTorch, was quantized, compiled with Vitis AI, and deployed to the FPGA via PYNQ. We deeply analyzed different DPU configurations and frequencies, focusing on resource utilization, power, FPS, and energy efficiency. Results show that resource utilization remains constant across frequencies, but lower frequencies increase energy consumption. For optimal performance and energy efficiency, high-MAC DPU configurations should be used at higher frequencies. To our knowledge, no prior research has fully detailed the Vitis TRD flow within Vitis AI. Most rely on Vivado TRD, which requires PetaLinux. This work offers a comprehensive guide for deploying AI models on FPGAs using Ubuntu, eliminating the need for PetaLinux expertise.**

***Index Terms*—FPGA, DPU, MPSoC, AI accelerator, Vitis AI, PYNQ, Vitis TRD, Kria KV260**

I. Introduction

Recent advancements in deep learning have transformed real-time applications requiring high performance and low power consumption. As neural networks become more complex, the demand for optimized hardware, particularly FPGA-based systems, has surged. FPGAs with DPUs (Deep Learning Processing Unit) offer customizable hardware acceleration for AI models [3], [5]. While many studies use Vitis AI with DPUs, most rely on the Vivado TRD (Targeted Reference Design) flow, which requires PetaLinux and hardware expertise [5], [6], [13]. Vitis TRD simplifies AI model deployment through a model-based design flow, and the PYNQ framework enables Python-based deployment, eliminating the need for low-level hardware knowledge [7], [12]. This research provides a practical guide for designing AI accelerators using Vitis AI, Vitis TRD, and PYNQ, along with a detailed analysis of how different DPU architectures, frequencies, and configurations impact the accelerator's performance, energy efficiency, and resource utilization.

II. Related Works

Few studies examine the impact of DPU architecture, configurations, and frequency adjustments using Vitis AI. Tatar and Bayar [14] evaluated a single DPU configuration for an ADAS model but tested only one parameter set, limiting the scope of their findings. In a later study [13], they tested seven configurations, but changing multiple parameters simultaneously made it difficult to isolate the effect of individual parameters, including frequency. Although they proposed configurations to enhance performance and energy efficiency, our findings indicate that some of the parameter sets they used are not well-suited for these objectives. Vandendriessche et al. [16] explored DPU architecture and frequency but with limited test sets, leaving some configuration parameters unexamined. Prakasham et al. [15] focused on DPU architecture's impact on resource utilization but did not address frequency or other configurations. Our study fills this gap by providing a comprehensive analysis of DPU architecture, configuration parameters, and frequency adjustments, detailing their influence on performance, energy efficiency, and resource utilization. To the best of our knowledge, no research thoroughly applies and explains the Vitis TRD flow within the context of Vitis AI. Most studies have used the Vivado TRD flow, which requires PetaLinux [13], [16]. Unlike the previous studies, this research offers a detailed guide for Vitis AI researchers working on Ubuntu without needing PetaLinux expertise.

III. Background Information

A. Model and Dataset

We used the ResNet18 CNN model, implemented in PyTorch, for vehicle color classification using the Vehicle Color Recognition (VCOR) dataset [1], [2]. ResNet18 was selected due to its lightweight design, making it efficient for deployment on resource-constrained hardware like FPGAs, while still providing strong image classification performance. It is also well-suited for fine-tuning with specific datasets, enabling relatively short training times. For fine-tuning, the VCOR dataset is used, sourced from Kaggle [2], consists of vehicle images labeled into 15 color classes.

B. Hardware Specifications

We used the AMD Kria KV260 Vision AI Starter Kit, a development platform designed to accelerate AI and vision applications [3]. This kit features the K26 SoM, which integrates the Zynq UltraScale+ MPSoC with 4GB of DDR memory [3]. The MPSoC combines a FPGA fabric which is programmable logic (PL) and a processing system (PS), which includes a

979-8-3503-9106-0/24/$31.00 © 2024 IEEE

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 13,2025 at 10:54:27 UTC from IEEE Xplore. Restrictions apply.

quad-core ARM Cortex-A53 for application processing and a dual-core ARM Cortex-R5F for real-time processing [3]. For this study, we installed Ubuntu 22.04 on a microSD card to provide a stable OS environment for the PS [4]. The PL side of the MPSoC hosts various FPGA resources that can be configured for tasks such as AI inference, signal processing, and data acceleration [3]. At the heart of AI acceleration is the DPU, specifically the DPUCZDX8G model, a specialized IP core implemented within the programmable logic of the FPGA [5]. The DPU interacts with the processing system via an AXI interface and provides flexible configuration options for resource usage, performance and parallelism [5]. The DPU supports variants ranging from B512 to B4096, indicating the number of MACs (multiply-accumulate operations) it can perform per clock cycle [6]. The DPUCZDX8G is optimized for executing 8-bit integer models, aligning with the Vitis AI quantization process. It accelerates key operations such as convolution, ReLU, pooling, and fully connected layers, significantly enhancing the performance of deep learning inference tasks [5], [6].

C. Vitis AI

We used Vitis AI 3.0 for the quantization and compilation of the AI model. Vitis AI, developed by AMD Xilinx, is a framework designed for deploying AI models on FPGA and SoC hardware, offering tools for model optimization, quantization, and compilation to efficiently execute deep learning models on the DPU [6]. The Vitis AI quantizer converts floating-point models into 8-bit integer models, reducing model size and computational complexity while maintaining accuracy [6]. These quantized models use less memory and deliver higher speed and power efficiency compared to floating-point models. The quantizer supports various layers, including convolution, pooling, and fully connected layers, and is compatible with TensorFlow and PyTorch frameworks [6]. The Vitis AI compiler optimizes models for target hardware by transforming them into the .xmodel format for execution on the DPU [6]. Additionally, the Vitis AI library provides pre-built functions and APIs for deploying AI inference applications, simplifying integration with DPU-based acceleration [6].

D. PYNQ

We used the PYNQ 3.0 framework and PYNQ-DPU overlay for AI model deployment [7]. After installing Ubuntu 22.04 on the SD card, both were automatically set up on the Kria KV260 [4]. PYNQ, an open-source framework by AMD Xilinx, provides a Jupyter-based environment with Python APIs, simplifying the design of adaptive computing systems [7]. The PYNQ-DPU overlay enables efficient deep learning inference by using the DPU, allowing users to interact with the hardware through Python without needing to manage the complex hardware configuration details [8].

IV. Methodology

A. Vitis AI Quantization and Compilation

We initially trained the ResNet18 model by following the methods outlined in the Vitis AI PyTorch ResNet18 repository by AMD Xilinx [9]. The model was fine-tuned using the VCOR dataset, with images resized to match the expected input resolution of 224x224. After retraining, the best validation accuracy achieved was 88.65%, and the model was saved as a .pt file for Vitis AI quantization. In the quantization step, we loaded the trained model weights from the .pt file and applied Post-Training Quantization (PTQ) [6]. The *torch_quantizer* from the Vitis AI library *pytorch_nndct.apis* was used to convert the model's weights and activations from 32-bit floating-point (FP32) to 8-bit integers (INT8) for deployment on the DPU [6]. Calibration was performed using the VCOR validation dataset to gather statistics on the range of the model's weights and activations, while the same dataset was used for testing and accuracy evaluation. After quantization, the model achieved an accuracy of 88.30%, a slight drop from 88.64%, with an average loss of 0.4665, indicating minimal performance degradation. The quantized model was exported as an .xmodel file. In the Vitis AI compilation process, this .xmodel was optimized for execution on the FPGA's DPU using the *vai_c_xir* command, which generates a compiled .xmodel based on the DPU architecture defined in the *arch.json* file, obtained through the Vitis TRD flow [5], [6].

B. Hardware Platform and Vitis TRD Flow

We used Vivado 2022.2 and Vitis 2022.2 to create the hardware design, both compatible with Vitis AI 3.0. The Vitis Targeted Reference Design (TRD) flow was followed [5] instead of Vivado TRD, which requires Petalinux. First, we created an RTL project in Vivado as an extensible Vitis platform [10], defining the base hardware block design with the Zynq Ultrascale+ MPSoC, a clocking wizard for DPU, DSP, and other blocks, and processor system resets for each output clock as shown in Figure 1. The AXI Full Power Domain (FPD) pins were enabled for high-speed access to DDR memory, while the Low Power Domain (LPD) pins were disabled, as AXI FPD is better suited for the DPU's performance and data throughput requirements. We set the DPU clock to 300 MHz (near its maximum), the DSP clock to 600 MHz (which should be twice the DPU clock), and a 100 MHz default clock for other blocks [5]. Each clock was assigned an ID and tag for use in the Vitis flow, and the platform design was exported as an .xsa file [10].

Next, we created a Vitis platform project. Since no default embedded base platform exists for KV260, we downloaded *kv260_ispMipiRx_vcu_DP* from the KV260 repository and built the platform using the Makefile [11]. After updating the hardware specifications with our .xsa file, the custom platform became ready. We then created a Vitis application project, following the Zynq UltraScale+ MPSoC DPU TRD V4.1 Vitis 2022.2 guide [12]. A DPU configuration set was defined in the *dpu_conf.vh* file [5], choosing the B4096 architecture (4096 MACs per clock cycle) and applying Configuration Set-1 as shown in Table 1. The *arch.json* file was automatically modified based on this configuration [5]. We defined the clocks and DPU connectivity in the *prj_config_gui* file using the IDs and tags previously assigned during the Vivado setup. After

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 13,2025 at 10:54:27 UTC from IEEE Xplore. Restrictions apply.

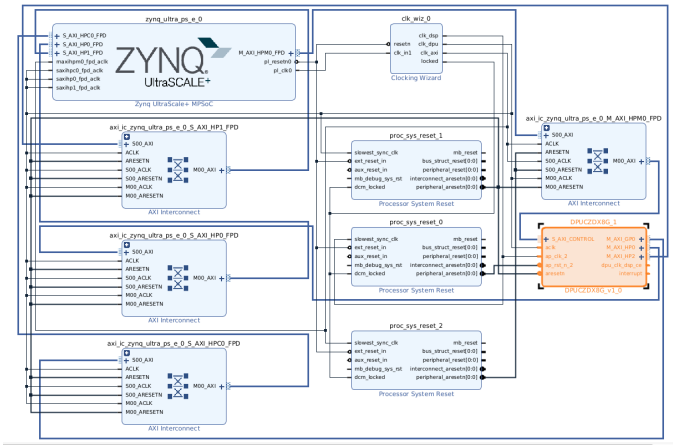

Fig. 1. The Hardware Design after Vitis TRD Flow.

building, we obtained the bitstream file (.bit), hardware design file (.hwh), and device binary file (.xclbin), which includes the compiled hardware design. These files were necessary and used for FPGA deployment [5].

C. Model Deployment in FPGA

In the model deployment step, we used the PYNQ framework and PYNQ-DPU overlay. Using the *DpuOverlay* class from the PYNQ-DPU package, the .bit file was loaded into the FPGA via the .hwh and .xclbin files [8]. The *load_model* method of the *DpuOverlay* class was used to load the previously compiled .xmodel file into the FPGA's DPU [8]. Images from the VCOR test dataset were loaded, resized, normalized, and prepared for inference using Python libraries like OpenCV and NumPy. This preprocessing ensured that the input data was in the correct format and size for the ResNet18 PyTorch model. The *runner* method from the Vitis AI Runtime (VART) [6] within the PYNQ-DPU overlay provided an interface to execute deep learning tasks on the DPU. We initialized the DPU runner, determined the input/output tensor shapes, and allocated memory accordingly. After loading the preprocessed images and running inference on the DPU, we calculated the model's accuracy and throughput (FPS). The initial design achieved an accuracy of 88.30%, consistent with the quantized model, and a throughput of 162.35 FPS. These results confirm that inference accuracy was maintained as expected, and the throughput was well-suited for most real-time applications.

V. Results and Discussion

The DPUCZDX8G IP allows users to configure various parameters to customize features such as DPU architecture, DSP usage, ReLU type, RAM usage, Channel Augmentation, Save Argmax, URAM usage, ALU parallelism, low-power, and Softmax. We tested a total of 40 different configurations to assess the impact of these parameters on performance, resource utilization, power, and energy consumption. Initially, we modified one parameter at a time while keeping the others constant to isolate its effect, as shown in Table 1. We selected the B4096 architecture, which offers the highest MACs per cycle, and set the DPU clock to 300 MHz, near its maximum frequency. Our findings are summarized as follows: Reducing DSP usage from high to low increases resource utilization but does not affect performance (See Set-1 and Set-2), highlighting the importance of using high DSP for efficient resource usage. Since the ResNet-18 model does not use Leaky ReLU, disabling it did not impact results significantly (See Set-1 and Set-3). However, for models using Leaky ReLU, this option should remain enabled [5]. Setting RAM usage to high for the B4096-300MHz configuration caused timing errors due to high resource consumption (See Set-4). For other DPU architectures, high RAM usage increased FPS, resource utilization, and power consumption, but still did not match the performance of the B4096-300MHz configuration. Disabling Channel Augmentation resulted in a significant drop in FPS and increased energy and power consumption. This option should be enabled to maximize parallelism, especially when the number of input channels is lower than available parallel channels [5]. Enabling the low-power option improved results without sacrificing performance (See Set-1 and Set-9). The Softmax option, which adds a hardware Softmax accelerator within the DPU IP wrapper, increased resource utilization, power, and energy consumption while slightly reducing FPS (See Set-1 and Set-10). If the model does not use a Softmax layer, this option should be disabled. Recommended values for URAM usage and ALU parallelism are outlined in the AMD PG338 guide [5]. In the B4096 configuration, recommended URAM usage is 46, though 64 URAMs are available. Tatar and Bayar [13] utilized full URAM capacity in some configurations to enhance performance, but our results showed that recommended usage provided better FPS without significant changes in resource utilization, power, or energy consumption (See Set-1 and Set-7). Similarly, while Tatar and Bayar [14] employed ALU parallel-8 for higher performance, we observed that the recommended ALU parallel setting resulted in better resource utilization and lower power and energy consumption, with only a minor reduction in FPS (See Set-1 and Set-8). Most importantly, for the B512 configuration, the recommended ALU parallel setting yielded better results, especially with an increase in FPS. That is, our findings show that higher URAM usage and maximum ALU parallel settings do not consistently lead to better performance across all DPU architectures.

After examining the impact of configuration parameters, we analyzed the effects of different DPU architectures and frequencies. We tested four architectures—B4096, B3136, B1024, and B512—each at five different frequency levels. For each test, we used the recommended URAM and ALU parallel values specific to each architecture while keeping the other parameters consistent with Set-1. Figure 2 visualizes these test sets with different shapes for each architecture and colors for each frequency. Our tests revealed that lowering the frequency for a given DPU architecture reduced power consumption but did not contribute to reducing resource utilization. Notably, as the frequency decreased, FPS dropped significantly, and energy consumption increased. The left plot in Figure 2 shows that utilization remains stable across frequencies for the same architecture, but higher frequencies lead to better performance.

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 13,2025 at 10:54:27 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DPU4096-300MHz CONFIGURATION TABLE WITH HIGHLIGHTED DIFFERENCES

Set Number	1	2	3	4	5	6	7	8	9	10
DPU Architecture	B4096									
Frequency	300 MHz									
DSP usage	high	low	high	high	high	high	high	high	high	high
LeakyReLU	enable	enable	disable	enable	enable	enable	enable	enable	enable	enable
RAM usage	low	low	low	high	low	low	low	low	low	low
Channel Augmentation	enable	enable	enable	enable	disable	enable	enable	enable	enable	enable
Save Argmax	enable	enable	enable	enable	enable	disable	enable	enable	enable	enable
URAM usage	46	46	46	46	46	46	64	46	46	46
ALU Parallel	4	4	4	4	4	4	4	8	4	4
Low-power	enable	enable	enable	enable	enable	enable	enable	enable	disable	enable
Softmax	disable	disable	disable	disable	disable	disable	disable	disable	disable	enable
Power (Watt)	6.462	6.36	6.458	6.863	6.507	6.504	6.36	6.804	6.501	6.907
FPS	162.35	162.18	162.09	Timing	145.08	162.02	160.02	163.27	162.17	162.03
Energy per frame (J)	0.0398	0.0392	0.0398	Error	0.0449	0.0401	0.0397	0.0417	0.0401	0.0426
Utilization(%)	0.394	0.422	0.391	0.409	0.391	0.393	0.395	0.447	0.397	0.443

As utilization increases from B512 to B4096, FPS also rises. The right plot demonstrates the inverse relationship between FPS and energy per frame—lower frequencies increase energy consumption per frame, as shown by the black markers on the lower right side for lower-frequency configurations like B512-100MHz. The results show that B4096 and B3136 at 300MHz provide the best balance between performance and energy efficiency. In contrast, low-frequency configurations are less efficient in terms of FPS, consume more energy per frame, and do not contribute to reducing resource consumption. Therefore, B4096-300MHz and B3136-300MHz are ideal for applications requiring high FPS and energy efficiency, while lower frequencies should be avoided in energy-sensitive designs.

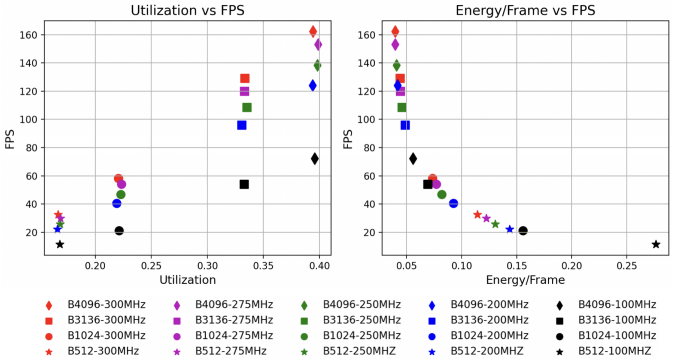

Fig. 2. Results for different DPU architecture and frequency

VI. CONCLUSION

Our model-based AI accelerator design, using the Vitis TRD flow for hardware deployment, along with Vitis AI for quantization and compilation, and PYNQ for deployment, offers an efficient and user-friendly approach to deploying AI models on FPGAs. We analyzed DPU configurations across different frequencies and architectures to evaluate their impact on power, energy consumption, FPS, and resource utilization. The results indicate that DPU architectures with higher MAC counts, such as B4096 and B3136 at 300MHz, offer the best balance between performance and energy efficiency. Lower frequencies increase energy consumption and reduce FPS drastically, while resource utilization remains constant. For applications requiring both high performance and energy efficiency, higher-frequency configurations with more MAC operations prove to be the optimal choice.

REFERENCES

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.

[2] L. Kezebou, "VCOR: Vehicle Color Recognition Dataset," Kaggle, [Online]. Available: https://www.kaggle.com/datasets/landrykezebou/vcor-vehicle-color-recognition-dataset. Accessed: August 2024.

[3] AMD Xilinx, "Kria KV260 Vision AI Starter Kit," *Product Documentation*, v2022.1, [Online]. Available: https://xilinx.github.io/kria-apps-docs/kv260/2022.1/build/html/index.html. Accessed: August 2024.

[4] AMD Xilinx, "Setting Up the SD Card Image," [Online]. Available: https://www.amd.com/en/products/system-on-modules/kria/k26/kv260-vision-starter-kit/getting-started-ubuntu/setting-up-the-sd-card-image.html. Accessed: August 2024.

[5] AMD Xilinx, "DPUCZDX8G for Zynq UltraScale+ MPSoCs Product Guide (PG338)," *Product Guide*, v2022.2, Jan. 2023. [Online]. Available: https://docs.amd.com/r/en-US/pg338-dpu. Accessed: August 2024.

[6] AMD Xilinx, "Vitis AI User Guide (UG1414)," v3.0, 2023. [Online]. Available: https://docs.amd.com/r/3.0-English/ug1414-vitis-ai/Overview. Accessed: August 2024.

[7] PYNQ Documentation, "Python Productivity for Zynq," [Online]. Available: https://pynq.readthedocs.io/en/latest/. Accessed: August 2024.

[8] Xilinx, "DPU-PYNQ," GitHub Repository, [Online]. Available: https://github.com/Xilinx/DPU-PYNQ/tree/master. Accessed: August 2024.

[9] AMD Xilinx, "Vitis AI Tutorials: PyTorch-ResNet18," GitHub Repository, [Online]. Available: https://github.com/Xilinx/Vitis-AI-Tutorials/tree/3.0/Tutorials/PyTorch-ResNet18. Accessed: August 2024.

[10] AMD Xilinx, "Vitis Tutorials: Vitis Platform Creation," AMD Documentation, [Online]. Available: https://docs.amd.com/r/2022.2-English/Vitis-Tutorials-Vitis-Platform-Creation. Accessed: August 2024.

[11] AMD Xilinx, "Kria Vitis Platforms: kv260_ispMipiRx_vcu_DP," GitHub Repository, [Online]. Available: https://github.com/Xilinx/kria-vitis-platforms/tree/xlnx_rel_v2022.2/kv260/platforms/vivado/kv260_ispMipiRx_vcu_DP. Accessed: August 2024.

[12] AMD Xilinx, "Vitis AI: DPUCZ Vitis Reference Design Documentation," GitHub Repository, [Online]. Available: https://github.com/Xilinx/Vitis-AI/blob/v3.0/dpu/ref_design_docs/README_DPUCZ_Vitis.md. Accessed: August 2024.

[13] G. Tatar and S. Bayar, "Energy efficiency assessment in advanced driver assistance systems with real-time image processing on custom Xilinx DPUs," *Journal of Real-Time Image Processing*, vol. 21, pp. 157, 2024, doi: 10.1007/s11554-024-01538-1.

[14] G. Tatar and S. Bayar, "Real-Time Multi-Task ADAS Implementation on Reconfigurable Heterogeneous MPSoC Architecture," *IEEE Access*, vol. 11, 2023.

[15] V. T. Prakasham, S. A. Saathappan, T. K. Harishankhar, R. Shailesh, and N. Mohan, "Improved Hardware Accelerator for Object Detection," in *IEEE International Conference on Signal Processing, Computation, Electronics, Power, and Telecommunication (ICONSCEPT)*, 2024.

[16] J. Vandendriessche, B. da Silva, and A. Touhafi, "Frequency Evaluation of the Xilinx DPU Towards Energy Efficiency," in *2022 Annual Conference of the IEEE Industrial Electronics Society (IECON)*, Brussels, Belgium, 2022, pp. 1-6. DOI: 10.1109/IECON49645.2022.9968811.

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 13,2025 at 10:54:27 UTC from IEEE Xplore. Restrictions apply.